

# U1T02: Document Intelligence

## Asynchronous Pipeline

Real services rarely process a request synchronously, file ingestion especially. Documents arrive faster than they can be processed; processing times depend on several factors such as file format and size, and users can't hold a connection open while heavy processes need to run. A common reliable solution is to accept the upload, store it, queue the work, and let users track progress asynchronously.

In this assignment, you will build a document intelligence service where users push documents, and through asynchronous processing, they get the contents from that document as plain text.

### Part 1: Ingestion and storage

Build a service exposing an endpoint to push documents. Accept at least PDF, PGN/JPG, and TXT file formats. Use the claim-check pattern architecture. Reject unsupported formats.

Add support for all additional file formats you want. Determine if any additional parameter as payload may add value to the pipeline. Determine reasonable upload limits.

### Part 2: Job tracking

Expose endpoints for job tracking so users can pull progress and results from the service. Add a database layer to track document lifecycle and store relevant job information.

### Part 3: Document Intelligence

Identify the best methods to parse documents content into machine readable text. Support your parser choices with a small comparison or benchmark, not just "I picked pytesseract".

Identify how and where to store extracted text.

Identify which other information may be worth extracting as metadata if any.

Use on-device technologies, use open-source tech as much as possible.

### Part 4: Queue

Recommended using Redis + Celery to define the architecture for the pipeline.

You are open to choosing other services, adding additional components or logic as you want.

## Part 5: Failure handling

The system must survive edge scenarios: unsupported formats, corrupted files, and workers (or the whole stack) being killed and restarted mid-processing; no job may be lost and no document may stay in a non-terminal state forever.

Add all necessary warnings, error messages, and status information so the user can understand what is happening with the system during normal usage or failure.

## Part 6: Monitoring (optional)

Add Celery Flower to track queue tasks

Create a dashboard to monitor tasks or whatever information you consider is worth monitoring.

Create a simple UI to upload documents and read results.

## Deliverables

- Code (zip, GitHub repo, or else). It all should run with a single docker compose command.
- Report: PDF containing architecture diagrams, product information, supporting arguments for the technical decisions made about the system.

## Notes

Here is an example of the Redis + Celery queue used in class:

<https://github.com/DexterGomez/queue-demo>

Claim-check patter: <https://commons-os.github.io/patterns/claim-check-pattern/>